

Le Conservatoire National des Arts et Métiers

Rapport sur la reproductibilité d'un article scientifique sur l'analyse des données et les probabilités

Fouille de données

30 mai 2026 par

Arsène MBABEH - Raphaël PÉNÉ - Oscar JOSEPH-GENESLAY

Promotion (2025-2026)

Titre : *Analyse de donnée et probabilité*

Table des matières

0.0.1	Package	1
1	Introduction	1
1.1	1. Probabilités Théoriques	1
1.1.1	Loi de Poisson	1
1.1.2	L'esperance mathématique	2
1.1.3	Application Shiny - La Loi Gamma	2
1.2	Lancement de l'application shiny	3
1.3	Analyse de Données : data = "mtcars"	3
1.3.1	Analyse exploratoire	3
1.4	Regréssion Linéaire	5
1.4.1	Interprétation :	6
1.5	Regression en Latex	7
1.5.1	Regression avec ggplot	7
1.6	Travail avec les données réel	8
1.6.1	Tableau des fréquences	9
1.6.2	La probabilité qu'une boule sorte	10
1.6.3	teste de khi deux	10
1.6.4	Simulation de Monté carlo	10

0.0.1 Package

```
### Installation des packages
#install.packages("dplyr")
#install.packages("kableExtra")
#install.packages("stargazer")
#install.packages("ggplot2")
#install.packages("ggthemes")
#install.packages("tidyr")
#install.packages("lubridate")
```

1. Introduction

Ce projet explore différents aspects de la statistique : de la théorie des lois de probabilité à l'analyse de données réelles (Keno) en s'inspirant de l'article (Evans, 2024).

1.1 1. Probabilités Théoriques

1.1.1 Loi de Poisson

La loi de Poisson est une loi de probabilité **discrète** qui décrit le nombre d'événements se produisant dans un intervalle de temps ou d'espace fixe.

Si $X \sim \mathcal{P}(\lambda)$, alors :

$$P(X = x) = \frac{\lambda^x}{x!} e^{-\lambda}, \quad x \in \mathbb{N}$$

```
x <- 0:10
dx <- dpois(x, lambda = 3)

plot(x, dx, type = "h", lwd = 2, col = "steelblue",
     main = expression("Loi de Poisson (" * lambda == 3 * ")"),
     xlab = "Nombre d'événements", ylab = "Probabilité")
points(x, dx, pch = 16, col = "steelblue")
```

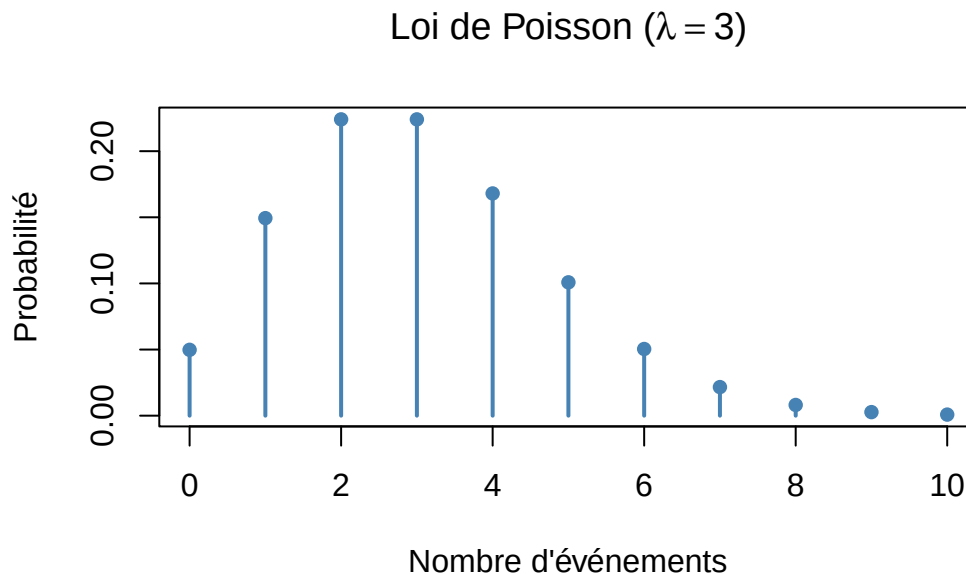


Figure 1: Distribution de Poisson

1.1.2 L'espérance mathématique

L'espérance mathématique, c'est la **valeur moyenne théorique** d'une variable aléatoire

```
# L'espérance c'est la somme des x * P(X=x)
esperance <- sum(x * dx)
print(paste("Espérance = ", round(esperance, 3)))
```

```
[1] "Espérance = 2.997"
```

$$\approx 3$$

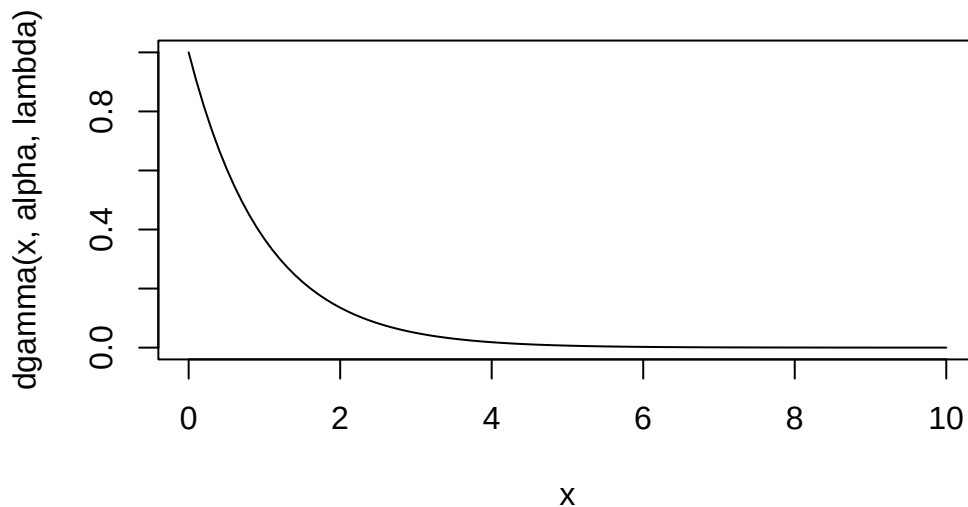
On peut aussi exécuter le chunk en ligne pour rechercher le résultat qui est ici : 2.997

1.1.3 Application Shiny - La Loi Gamma

**** Rappel ** :**

La loi de Gamma est une distribution de probabilité continue qui modélise des variables aléatoires réelles positives. Elle est utilisée pour décrire divers phénomènes, tels que la durée de vie d'un système ou le temps entre des événements.

```
alpha = 1
lambda = 1
curve(dgamma(x,alpha,lambda), 0, 10)
```



1.2 Lancement de l'application shiny

<https://lapoulefolle64.shinyapps.io/Projet-Killani><empty citation>/

1.3 Analyse de Données : data = “mtcars”

1.3.1 Analyse exploratoire

Tout d'abord on commence par afficher les noms des colonnes du dataset avec la fonction **names()**, puis on crée un résumé statistique, de tel sorte à obtenir en sortie un tableau présentant des stats (moyenne et écart-type) des variables mpg et cyl

```
library(dplyr)
library(tidyverse)
voiture <- mtcars

voiture|>names() ### pour voir le nom de chaque colonne
```

```
[1] "mpg" "cyl" "disp" "hp" "drat" "wt" "qsec" "vs" "am" "gear"
[11] "carb"
```

```
voiture |> select(mpg, cyl) |> summarise(mean_mpg = mean(mpg), mean_cyl = mean(cyl), sd_mpg = sd(mpg), sd_cyl = sd(cyl))
```

On transforme les résultats statistiques en une matrice structurée, en entrant manuellement les valeurs (moyenne, écart-type)

```
library(dplyr)
library(kableExtra)
```

Attaching package: 'kableExtra'

The following object is masked from 'package:dplyr':

group_rows

```
library(tidyverse)

stat_desc |> matrix(c(20.02, 6.19, 6.03, 1.79), nrow = 2, ncol = 2,
                  dimnames = list(c("mean", "sd"), c("mpg", "cyl"))) |> kable() -> matrice

matrice
```

	mpg	cyl
mean	20.09	6.19
sd	6.03	1.79

```
library(dplyr)
library(kableExtra)
library(tidyverse)

# stat desc
stat_desc <- mtcars |>
  select(mpg, cyl) |>
  summarise(
    across(everything(), list(mean = mean, sd = sd))
  ) |>
  round(2)

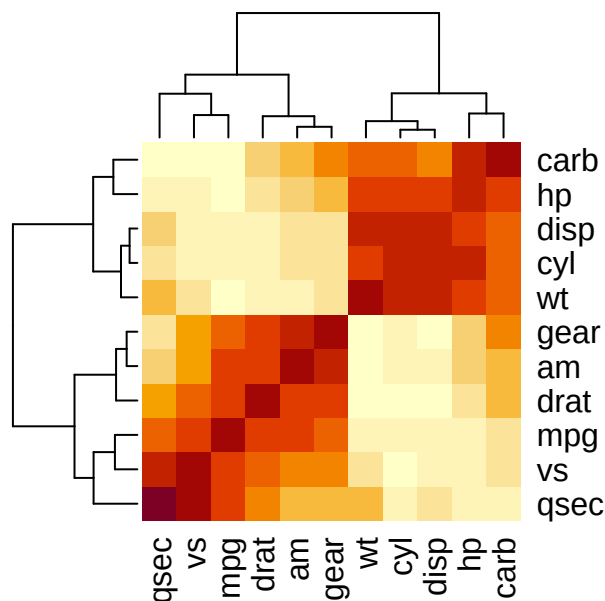
# présentation sous forme de tableau???
kable(stat_desc, caption = "Moyenne et Écart-type (MPG et Cylindres)") |>
  kable_styling(bootstrap_options = "striped")
```

Table 1: Moyenne et Écart-type (MPG et Cylindres)

mpg_mean	mpg_sd	cyl_mean	cyl_sd
20.09	6.03	6.19	1.79

Nous allons ensuite analyser les corrélations entre variables, afin d'identifier si des relations linéaire existe dans notre jeu de donnée mtcars

```
correlation_matrix <- cor(mtcars)
heatmap(correlation_matrix)
```



On constate une corrélation assez faible entre la puissance et la consommation

1.4 Régression Linéaire

Nous cherchons à expliquer la consommation (mpg) par la puissance (hp), en utilisant le jeu de donnée mtcars

```
library(stargazer)
```

Please cite as:

Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.

R package version 5.2.3. <https://CRAN.R-project.org/package=stargazer>

```
library(ggplot2)
library(ggthemes)

modele <- lm(mpg ~ hp, data = mtcars)

# Tableau de régression
summary(modele, title = "Impact de la puissance (hp) sur la consommation (mpg)")
```

Call: lm(formula = mpg ~ hp, data = mtcars)

Residuals: Min 1Q Median 3Q Max -5.7121 -2.1122 -0.8854 1.5819 8.2360

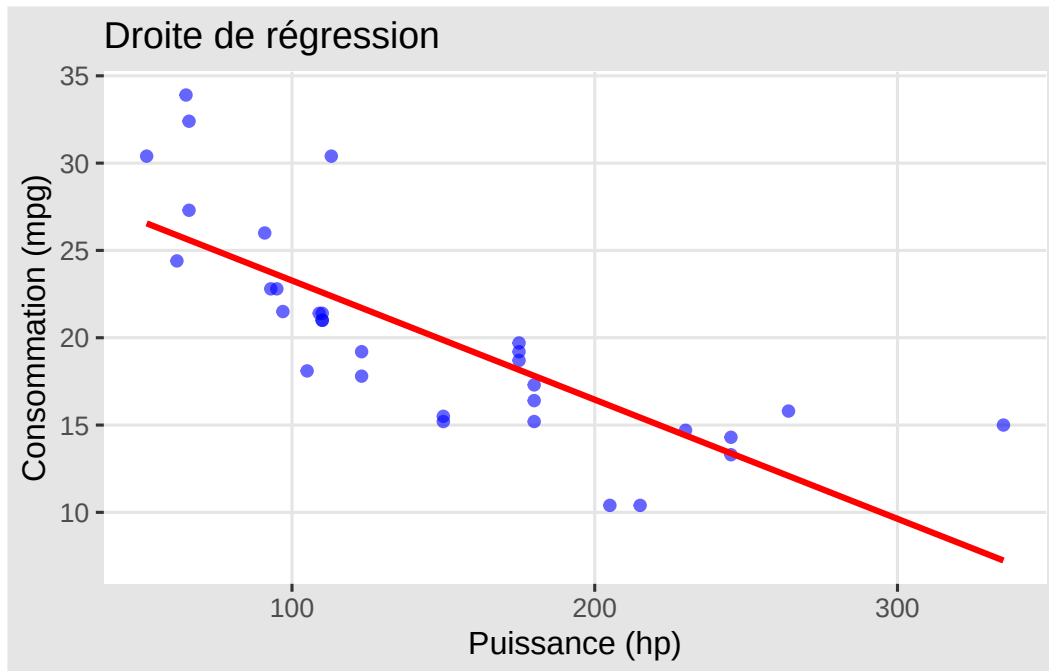
Coefficients: Estimate Std. Error t value Pr(>|t|)

(Intercept) 30.09886 1.63392 18.421 < 2e-16 **hp -0.06823 0.01012 -6.742 1.79e-07** — Signif. codes: 0 ‘**0.001**’ 0.01 ‘**0.01**’ 0.05 ‘.’ 0.1 ‘.’ 1

Residual standard error: 3.863 on 30 degrees of freedom Multiple R-squared: 0.6024, Adjusted R-squared: 0.5892 F-statistic: 45.46 on 1 and 30 DF, p-value: 1.788e-07

```
# Visualisation
ggplot(mtcars, aes(x = hp, y = mpg)) +
  geom_point(col = "blue", alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE, col = "red") +
  theme_igray() +
  labs(title = "Droite de régression", x = "Puissance (hp)", y = "Consommation (mpg)")
```

```
`geom_smooth()` using formula = 'y ~ x'
```



1.4.1 Interprétation :

Le modèle de régression linéaire utilisé est le suivant : $mpg \sim hp$, ce qui signifie que la variable "mpg" est expliquée en fonction de la variable hp

- L'intercept : son estimation est de 30.09, ce qui signifie que lorsque la variable hp est égale à zéro, la variable mpg est estimée à 30.09
- Le coefficient de hp : son estimation est de -0.068, ce qui signifie que pour chaque unité d'augmentation de hp, la variable mpg diminue en moyenne de 0.068 unités

Le R^2 est de 0.60, ce qui signifie que le modèle explique environ 60% de la variance dans la variable mpg, autrement dit la 60% de la variabilité de la mpg est expliqué par notre modèle

En bref, notre modèle met en évidence le fait que la variable hp a un effet significatif sur la variable mpg. Pour chaque augmentation d'une unité de hp, la valeur de mpg va diminuer

$$\approx 0.068 \text{ unit}$$

unités

1.5 Regression en Latex

```
library(stargazer)

modele = lm(mpg ~ hp, data = voiture)

### Pour faire le tableau
stargazer(modele,
  type = "latex",
  title = "Modèle de régression du mpg expliqué par le hp",
  dep.var.labels = "mpg",
  covariate.labels = "hp",
  digits = 3)
```

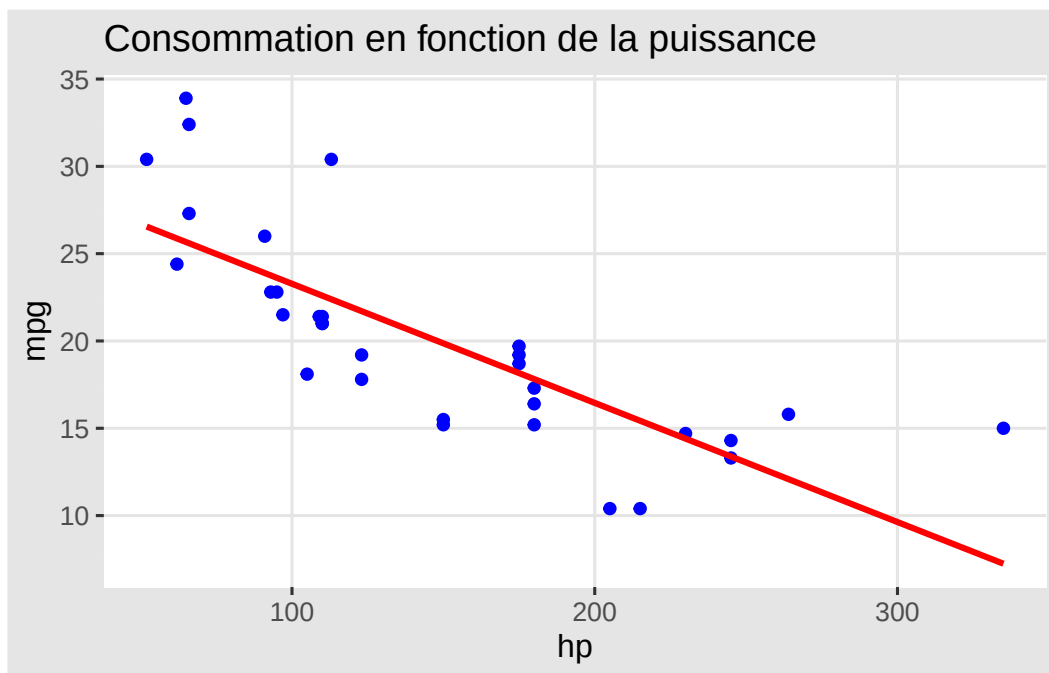
% Table created by stargazer v.5.2.3 by Marek Hlavac, Social Policy Institute. E-mail: marek.hlavac at gmail.com % Date and time: Tue, May 26, 2026 - 06:14:22 PM

Table 2: *Modèle de régression du mpg expliqué par le hp*

<i>Dependent variable:</i>	
	mpg
hp	−0.068*** (0.010)
Constant	30.099*** (1.634)
Observations	32
R ²	0.602
Adjusted R ²	0.589
Residual Std. Error	3.863 (df = 30)
F Statistic	45.460*** (df = 1; 30)
<i>Note:</i> *p<0.1; **p<0.05; ***p<0.01	

1.5.1 Regression avec ggplot

```
voiture |> ggplot(aes(hp, mpg)) + theme_igray() + ggtitle("Consommation en fonction de la puissance") +
  `geom_smooth()` using formula = 'y ~ x'
```



1.6 Travail avec les données réel

On importe les données historiques du jeu Keno, puis on transforme les données du format **large** (c'est à dire une une colonne par boule) au format **long** pour faciliter notre analyse

```
library(tidyr)
library(lubridate)
library(tidyr)
library(dplyr)

df = read.csv2("keno_202511 6 (1).csv")

## on supprime des variable
df = df[,-c(1, 3,20,21,22,23)]

# On convertit la date PUIS on bascule au format long
df1 <- df |>
  mutate(date_de_tirage = dmy(date_de_tirage)) |>
  pivot_longer(
    cols = !date_de_tirage,
    names_to = "boule",
    values_to = "valeur",
    names_prefix = "boule"
  )

df1

# A tibble: 1,600 x 3
  date_de_tirage boule valeur
  <date>         <chr> <int>
1 2026-02-10     1      12
```

```

2 2026-02-10      2      13
3 2026-02-10      3      17
4 2026-02-10      4      20
5 2026-02-10      5      22
6 2026-02-10      6      23
7 2026-02-10      7      26
8 2026-02-10      8      27
9 2026-02-10      9      39
10 2026-02-10     10      42
# i 1,590 more rows

```

```

# Vérification de la date la plus récente
max(df1$date_de_tirage)

```

```
[1] "2026-02-10"
```

1.6.1 Tableau des fréquences

```
library(scales)
```

Attaching package: 'scales'

The following object is masked from 'package:purrr':

```
discard
```

The following object is masked from 'package:readr':

```
col_factor
```

```

# Création du tableau des fréquences et pourcentages
tableau_frequences <- df1 |> group_by(valeur) |> summarise(Frequence = n()) |>
  mutate(Pourcentage = percent(Frequence / sum(Frequence), accuracy = 0.01))

tableau_frequences

```

```

# A tibble: 56 x 3
  valeur Frequence Pourcentage
  <int>   <int>   <chr>
1     1     30 1.87%
2     2     26 1.62%
3     3     28 1.75%
4     4     32 2.00%
5     5     17 1.06%
6     6     21 1.31%
7     7     37 2.31%
8     8     23 1.44%
9     9     33 2.06%
10    10     30 1.87%
# i 46 more rows

```

1.6.2 La probabilité qu'une boule sorte

1.6.3 teste de khi deux

Pour déterminer si les écarts de fréquences observés sont dus au hasard ou à un biais, nous réalisons un test de conformité à une loi uniforme, en fait on va chercher à vérifier l'équiprobabilité, c'est à dire qu'on se pose la question de savoir si toutes les boules ont la même probabilité d'être tirée

Sous l'hypothèse

- H_0 : Les boules ont toutes la même probabilité de sortir
- H_1 : Les boules n'ont pas du tout la même probabilité de sortir

NB : Notons que si la $p_value < \alpha$: on rejette l'hypothèse nulle

```
chisq.test(tableau_frequencies$Frequence)
```

Chi-squared test for given probabilities

```
data: tableau_frequencies$Frequence
X-squared = 45.14, df = 55, p-value = 0.826
```

Le test de khi deux sortant une p-value de 0.82, on conserve l'hypothèse d'équiprobabilité, donc aucune boule n'est statistiquement favorisée

1.6.4 Simulation de Monté carlo

Afin de renforcer notre conclusion du khi deux, nous simulons 10 000 fois une série de 100 jours de tirages pour observer la distribution du minimum de fréquence (la boule qui sort le moins souvent).

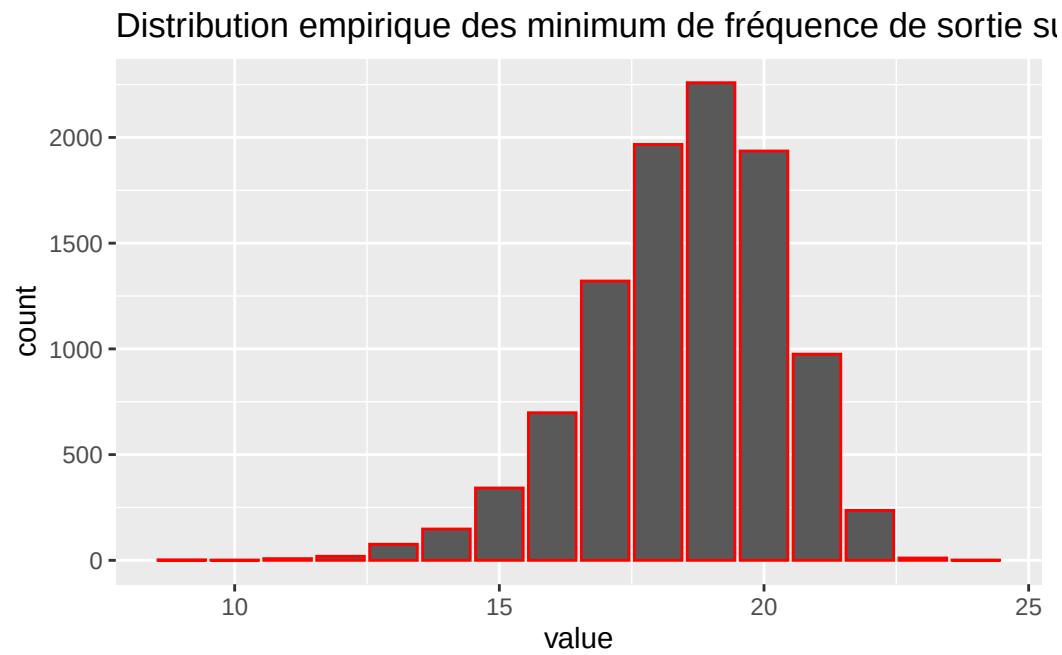
```
set.seed(123)
res <- c()
for (i in 1:10000) {
  # on simule en gros 100 jours de tirages (16 boules parmi 56)
  ech <- replicate(100, sample(1:56, 16, replace = FALSE))
  res <- c(res, min(table(ech)))
}

min(res)
```

```
[1] 9
```

```
library(ggplot2)

as_tibble(res) |> ggplot(aes(value)) + geom_bar(col = "red") + ggtitle("Distribution empirique des
```



```
as_tibble(res) |> filter(value <= 17) |> summarise(Frequence = n())
```

```
# A tibble: 1 x 1
```

```
  Frequence
```

```
    <int>
```

```
1       2615
```

p value est forte donc on conserve l'hypothèse selon laquelle aucune boule n'est laissée

References

Evans, K. (2024). Innovative and interactive statistics teaching using quarto. *Teaching Statistics*.
<https://doi.org/10.1111/test.12409>